CHARUSAT®

CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF TECHNOLOGY AND ENGINEERING

DEVANG PATEL INSTITUTE OF ADVANCE TECHNOLOGY AND RESEARCH

DEPARTMENT OF COMPUTER ENGINEERING

A.Y 2026 – 27 Semester – 3

LAB MANUAL

CSUC201 Fundamentals of Data Structures and Algorithms

DEPSTAR

Practical 6: Stack Implementation using Array and Linked List, with Expression Processing

Practical Aim: To implement stack using array and linked list, and apply stack-based logic to expression conversion, evaluation, and bracket validation problems.

Problem 1:

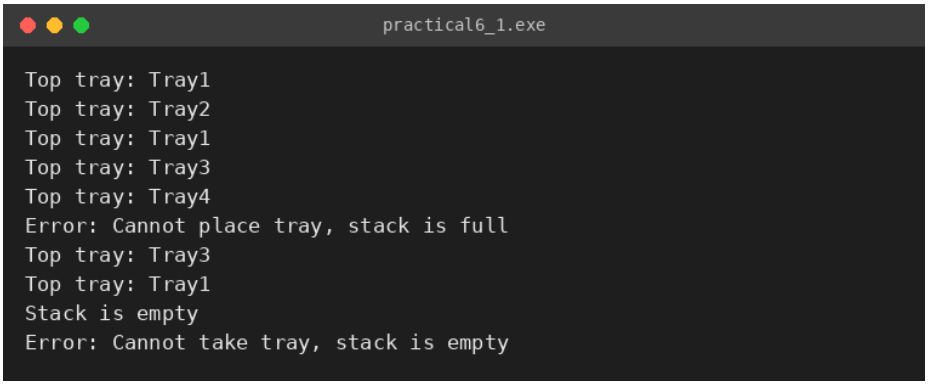
A cafeteria stacks clean trays on a fixed-size counter. New trays are always placed on top, and customers always take from the top. The counter can hold at most n trays at a time — if it is full, no more trays can be added, and if it is empty, no tray can be taken. Given a sequence of place and take operations, implement this fixed-capacity tray stack and print the current top tray after each operation. Report an error if a place or take operation cannot be performed.

Approach: The stack is stored in a std::vector (dynamic array) whose size is limited by the program to a capacity n. Before every place (push) the program checks whether size >= capacity; if so it prints "Error: Cannot place tray, stack is full" and does nothing. Before every take (pop) it checks whether the stack is empty; if so it prints "Error: Cannot take tray, stack is empty". Otherwise the top element is added or removed and the new top tray is printed.

For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-6/practical6_1.cpp

Output :



Problem 2:

A web browser keeps track of pages visited so that the back button always returns to the most recently visited page. Unlike a fixed counter, the browser has no hard limit on how many pages it can remember — it grows as the user visits more pages and shrinks as they press back. Given

a sequence of visit and back operations, implement this unlimited page history and print the current page after each operation.

Approach: Unlike Problem 1, there is no capacity check: every visit simply adds a page on top and the history grows dynamically as long as memory is available. Pressing back removes the top page and prints the page now on top. When the last page is removed the program prints "No pages left in history", and if back is pressed again with nothing left, it reports "Error: No pages in history" instead of crashing.

For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-6/practical6_2.cpp

Output :



Problem 3:

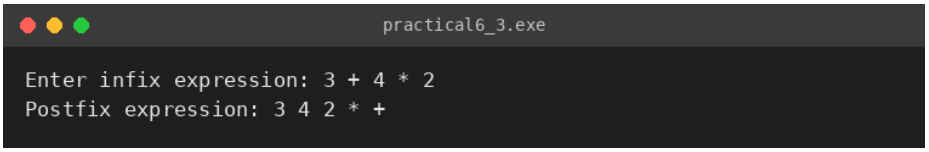
A calculator application receives arithmetic expressions typed by users in the usual way — numbers and operators written in between, with brackets to control priority (e.g. 3 + 4 * 2 or (3 + 4) * 2). Internally the calculator needs to convert each expression into a form where operators appear after their operands, so it can evaluate them without needing to re-read brackets. Given an infix expression, convert it to the equivalent postfix form and print the result.

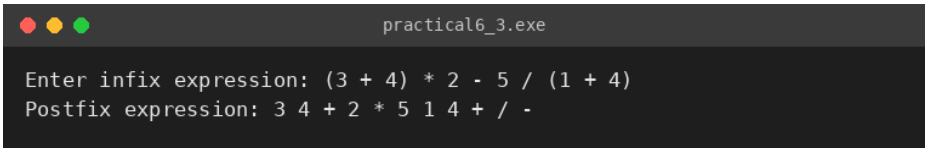
Approach: The expression is scanned from left to right. Operands go straight to the output. Operators and opening brackets are kept on a stack: before pushing an operator, every operator on top of the stack with higher or equal precedence is popped to the output first. On a closing bracket, the stack is popped to the output until the matching opening bracket is found, which is then discarded; this releases all operators that were waiting inside the brackets. At the end, the remaining operators are popped to the output.

For Code:

https://github.com/Aarav280907/FDSA_A2_SHP/blob/main/PRACTICAL-6/practical6_3.cpp

Output :





KEY QUESTIONS:

Problem 1: A circular array is one way to reuse freed slots without shifting all elements. Without implementing it, explain what change would need to be made to your front and rear pointer logic to make the array behave circularly, and what new condition would indicate a full array.

Problem 2: Your linked list stack has no fixed capacity limit. But in practice, can it still run out of space? What determines the actual limit, and why is that limit invisible in your code?

Problem 3: Operator precedence determines the order in which operators are placed into the output. Without tracing any expression, explain what would go wrong in your output if your solution treated all operators as having equal precedence.

ANSWERS:

ANS-1: To make an array circular, the front and rear indices must never simply grow forever; each time they move, they wrap around using the modulus operator: rear = (rear + 1) % capacity and front = (front + 1) % capacity. Freed slots at the beginning of the array are then reused instead of requiring all elements to be shifted.

The full condition changes accordingly. With a counter, the array is full when count == capacity. Without a counter, the array is full when (rear + 1) % capacity == front (one slot is left unused to tell 'full' apart from 'empty', where front == rear). A plain stack only needs a single top index because pops free the top slot immediately, so circular indexing matters mainly for queues (Practical 7).

ANS-2: Yes. Having no fixed limit in the code does not mean unlimited memory. Every push creates a new node on the heap, so the real limit is the memory the operating system can give to the program (physical RAM plus virtual memory, and any per-process limit). When it runs out, new fails (throws std::bad_alloc or returns null) and the program crashes or must handle the error.

The limit is invisible in the code because there is no capacity variable and no comparison against one; the program only asks the system for another node, and the hardware/OS limit appears only at the moment the allocation fails.

ANS-3: If all operators had equal precedence, the stack would pop an operator whenever another operator arrives, simply in left-to-right order. A higher priority operator such as * or / would no longer bind tighter than + or -, so the postfix output would apply operators in the order they were typed instead of the order mathematics requires.

For example, 3 + 4 * 2 should become 3 4 2 * + (value 11), but with equal precedence it would become 3 4 + 2 * (value 14). The calculator would therefore evaluate the expression incorrectly, even though the conversion itself looks valid.